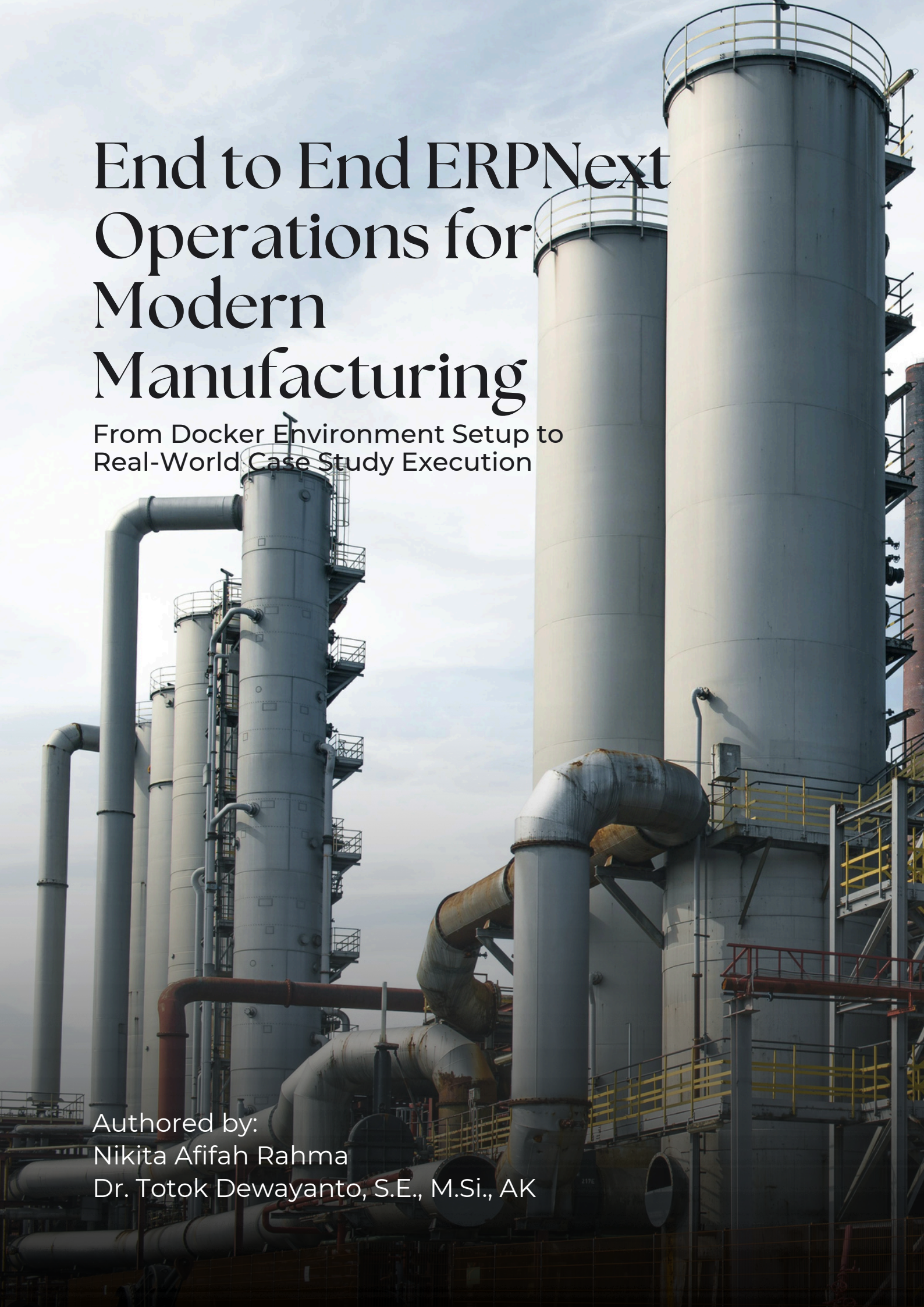


# End to End ERPNext Operations for Modern Manufacturing

## From Docker Environment Setup to Real-World Case Study Execution

Authored by:
Nikita Afifah Rahma
Dr. Totok Dewayanto, S.E., M.Si., AK

# End to End ERPNext Operations for Modern Manufacturing

From Docker Environment Setup to Real-World Case Study Execution

**Disusun oleh:**

**Penulis Pertama** **Nikita Afifah Rahma (12030123120064)**

**Penulis Kedua** **Dr. Totok Dewayanto, S.E., M.Si., Ak.**

**Program Studi Akuntansi**
**Universitas Diponegoro**
**2026**

# DAFTAR ISI

**DAFTAR ISI** ..... **2**

**BAB I**

**KONSEP DASAR ERPNEXT & FONDASI FRAPPE FRAMEWORK** ..... **4**

1.1 Konsep Dasar ERPNext ..... 4

1.2 Konsep Dasar Frappe Framework dalam Ekosistem ERPNext ..... 5

1.3 Arsitektur Frappe Framework ..... 7

**BAB II**

**INSTALASI & KONFIGURASI LINGKUNGAN FRAPPE** ..... **10**

2.1 Persiapan Lingkungan Virtualisasi Windows dan WSL 2 ..... 10

2.3 Pengunduhan dan Penyiapan Repositori frappe_docker ..... 11

2.4 Deployment Container ERPNext Menggunakan Docker Compose ..... 12

2.5 Pengaksesan Sistem dan Pengisian Setup Wizard ..... 13

2.6 Operasional Harian dan Prosedur Reset Sistem ..... 13

**BAB III**

**KONFIGURASI MASTER DATA PERUSAHAAN MANUFAKTUR** ..... **15**

3.1 Bagan Akun (Chart of Accounts/COA) Standar Manufaktur 4 Level ..... 15

3.2 Konfigurasi Master Gudang (Warehouse) dan Hirarki Lokasi ..... 16

3.3 Pengelolaan Hak Akses Pengguna dan Peran (Role & Permissions) ..... 17

3.4 Impor Masal Master Data Menggunakan Data Import Tool ..... 18

**BAB IV**

**OPERASIONAL ENTRY DATA TRANSAKSI BISNIS** ..... **20**

4.1 Modul Pembelian dan Pengadaan Barang ..... 20

4.2 Modul Penjualan dan Pengelolaan Pelanggan ..... 21

4.3 Modul Persediaan Stok dan Operasional Manufaktur ..... 22

4.4 Modul Akuntansi dan Keuangan Operasional ..... 23

4.5 Data Import Tool dan Pengunggahan Data Masal ..... 24

**BAB V**

**PEMBELIAN & PENERIMAAN BAHAN BAKU..... 26**

5.1 Penjelasan Cara Melakukan Fitur di ERPNext.....26

5.2 Studi Kasus & Latihan Praktikum..... 26

**BAB VI**

**PENGELUARAN BAHAN BAKU KE LINI PRODUKSI..... 28**

6.1 Penjelasan Cara Melakukan Fitur di ERPNext.....28

6.2 Studi Kasus & Latihan Praktikum..... 28

**BAB VII**

**PENCATATAN BIAYA TENAGA KERJA LANGSUNG & OVERHEAD PABRIK.....30**

7.1 Penjelasan Cara Melakukan Fitur di ERPNext.....30

7.2 Studi Kasus & Latihan Praktikum..... 30

**BAB VIII**

**PENYELESAIAN PRODUKSI & PENGAKUAN BARANG JADI..... 32**

8.1 Penjelasan Cara Melakukan Fitur di ERPNext.....32

8.2 Studi Kasus & Latihan Praktikum..... 32

**BAB IX**

**PENJUALAN BARANG JADI & PENGAKUAN HARGA POKOK PENJUALAN.....34**

9.1 Penjelasan Cara Melakukan Fitur di ERPNext.....34

9.2 Studi Kasus & Latihan Praktikum..... 34

**BAB X**

**PELUNASAN PIUTANG & JURNAL PENYESUAIAN PENYUSUTAN MESIN..... 36**

10.1 Penjelasan Cara Melakukan Fitur di ERPNext.....36

10.2 Studi Kasus & Latihan Praktikum..... 36

# BAB I
# KONSEP DASAR ERPNEXT & FONDASI FRAPPE FRAMEWORK

## 1.1 Konsep Dasar ERPNext

ERPNext adalah sistem *Enterprise Resource Planning* (ERP) terintegrasi berbasis *free* and *open-source* yang dikembangkan oleh Frappe Technologies Pvt. Ltd.. Sistem ini dibangun di atas Frappe Framework menggunakan bahasa pemrograman Python di sisi server, JavaScript/Vue.js di sisi *client*, serta basis data MariaDB. ERPNext dirilis di bawah lisensi terbuka GPL-3.0 tanpa biaya lisensi pengguna (*per-user license fee*), menjadikannya solusi efisien bagi skala Usaha Kecil dan Menengah (UMKM) hingga enterprise.

Secara fungsional, cakupan modul operasional ERPNext sangat menyeluruh untuk memenuhi kebutuhan transaksi harian maupun analitik jangka panjang. Modul akuntansi menangani buku besar, faktur, rekonsiliasi bank, dan laporan keuangan komprehensif, sementara modul inventaris mengelola keluar masuk barang di multi-gudang lengkap dengan pelacakan nomor seri, kedaluwarsa, dan metode valuasi stok. Pada saat yang sama, alur komersial dikawal melalui modul CRM dan penjualan untuk mengelola prospek hingga pesanan, yang kemudian diselaraskan dengan modul pembelian dan modul manufaktur guna merencanakan kebutuhan bahan baku, jadwal produksi, serta alokasi kapasitas kerja secara presisi.

Dalam operasional bisnis tradisional, setiap departemen sering kali bekerja secara terisolasi menggunakan spreadsheet terpisah, yang memicu duplikasi data, kesalahan input, dan hilangnya transparansi. ERPNext memposisikan dirinya sebagai pustaka data terpusat (*central brain*) dan satu-satunya sumber kebenaran (*single source of truth*) bagi seluruh organisasi. Setiap transaksi di satu modul (seperti penjualan atau pembelian) akan secara otomatis memperbarui saldo persediaan, akun keuangan, dan status operasional terkait di seluruh sistem secara *real-time*.

Selain aktivitas rantai pasok dan finansial, ERPNext juga mengintegrasikan fungsi pendukung penting seperti manajemen sumber daya manusia, pencatatan absensi, pengajuan cuti, pemrosesan slip gaji, penanganan tiket bantuan pelanggan, hingga pemantauan profitabilitas proyek berbasis lembar waktu. Kemudahan adaptasi menjadi kekuatan operasional lainnya, sebab pengguna dapat menyesuaikan formulir, menambah bidang data baru, menyusun alur

persetujuan, serta menghasilkan dasbor analitik visual secara langsung dari antarmuka web. Hasilnya, ERPNext berfungsi sebagai pusat kendali terpadu yang mendorong transparansi, efisiensi operasional, dan kepemilikan data bisnis secara penuh dan mandiri.

### 1.2 Konsep Dasar Frappe Framework dalam Ekosistem ERPNext

Frappe Framework merupakan kerangka kerja aplikasi web full-stack sumber terbuka berbasis Python dan JavaScript yang berperan sebagai fondasi arsitektur dan mesin penggerak di balik seluruh operasional ERPNext. Dikembangkan dengan filosofi metadata driven, Frappe memungkinkan perancangan aplikasi bisnis secara cepat dan terstruktur melalui pemanfaatan DocType sebagai entitas data inti. Melalui DocType ini, skema tabel pada basis data MariaDB atau PostgreSQL, antarmuka pengguna formulir di Frappe Desk, aturan hak akses pengguna, alur kerja dokumen, hingga ketersediaan REST API dapat didefinisikan secara deklaratif dalam satu konfigurasi terpadu tanpa harus membangun komponen teknis dasar tersebut dari nol.

Di atas fondasi Frappe Framework tersebut, berdiri ERPNext sebagai sebuah sistem perencanaan sumber daya perusahaan atau Enterprise Resource Planning yang utuh dan siap pakai. Sebagai perangkat lunak sumber terbuka berlisensi bebas biaya lisensi pengguna, ERPNext dirancang untuk mengintegrasikan seluruh operasional bisnis ke dalam satu basis data terpusat, menjadikannya alternatif mandiri yang kuat bagi berbagai skala usaha. Operasional yang dicakup meliputi pembukuan akuntansi dan rekonsiliasi keuangan, manajemen stok dan valuasi multi-gudang, pengelolaan hubungan pelanggan serta penjualan, pengadaan barang dari pemasok, perencanaan kebutuhan manufaktur, hingga administrasi kepegawaian serta penggajian.

Hubungan di antara keduanya dapat diibaratkan seperti fondasi bangunan dan rumah yang berdiri di atasnya, di mana Frappe berperan mengendalikan infrastruktur teknis dasar, sedangkan ERPNext menyediakan fungsionalitas logika bisnis bagi pengguna akhir. Keunggulan utama dari ekosistem ini terletak pada fleksibilitas kustomisasinya yang tinggi, memungkinkan penyesuaian alur kerja melalui skrip klien, skrip peladen, maupun penambahan kolom data baru tanpa mengubah kode inti aplikasi. Didukung tumpukan teknologi modern yang mengandalkan Linux, basis data MariaDB, Redis untuk antrean tugas, serta Nginx sebagai reverse proxy, ekosistem Frappe dan ERPNext menawarkan kendali penuh atas kepemilikan data dan efisiensi digitalisasi operasional bisnis secara menyeluruh.

Peran arsitektural Frappe yang fleksibel dan berorientasi data tersebut menemukan relevansi tertingginya ketika dihadapkan pada modul-modul fungsional ERPNext, terutama kegiatan akuntansi dan manajemen finansial. Karena akuntansi menuntut integritas data yang mutlak, kepatuhan regulasi, serta alur otorisasi yang ketat, modul akuntansi ERPNext tidak sekadar membutuhkan antarmuka pencatatan angka, melainkan sebuah kerangka kerja yang mampu menjamin konsistensi logika transaksi di setiap tahap pemrosesan. Di titik inilah Frappe Framework mengambil peran sentral bukan sebagai aplikasi pembukuan itu sendiri, melainkan sebagai mesin penjamin validitas teknis, keamanan, dan keandalan di balik setiap pergerakan debit dan kredit yang dicatat oleh sistem.

Secara teknis, Frappe menyediakan fondasi kepatuhan audit yang kukuh melalui mekanisme immutability dan penyerahan dokumen (document submission workflow) pada DocType akuntansi, seperti Sales Invoice, Payment Entry, maupun Journal Entry. Berkat arsitektur status dokumen bawaan Frappe, transaksi akuntansi yang telah berstatus submitted dikunci secara permanen di tingkat kerangka kerja untuk mencegah manipulasi data historis, sehingga setiap koreksi wajib dilakukan melalui pembatalan (cancellation) atau penerbitan dokumen pembalik (reversal entry) yang meninggalkan jejak audit (audit trail) lengkap secara otomatis. Selain itu, manajemen izin berbasis peran (role-based permission control) pada Frappe memastikan pemisahan tugas (segregation of duties) berjalan efektif, di mana hak akses untuk membuat draf faktur, menyetujui jurnal, hingga mengubah bagan akun (chart of accounts) dapat dibatasi secara presisi hingga ke tingkat baris maupun nilai nominal transaksi tertentu.

Frappe Framework bertindak sebagai lapis orkestrasi otomatisasi finansial dan integrasi pelaporan melalui logika peristiwa (document events) dan antrean tugas asinkron. Ketika suatu transaksi bisnis diproses di sisi klien, kait siklus hidup (controller hooks) di sisi peladen Python milik Frappe secara otomatis memicu kalkulasi pajak terpusat, konversi mata uang jamak, serta pembaruan saldo buku besar (General Ledger) dan piutang/utang secara real-time di dalam basis data. Untuk pelaporan finansial berskala besar seperti neraca keuangan dan laporan laba rugi yang memproses jutaan baris entri buku besar, Frappe memanfaatkan teknologi caching Redis dan arsitektur kueri SQL teroptimasi, memungkinkan ekspor data yang cepat, penyajian dasbor analitik visual tanpa jeda, serta pertukaran data keuangan ke perangkat lunak perpajakan atau perbankan pihak ketiga secara aman melalui REST API bawaan.

### 1.3 Arsitektur Frappe Framework

Frappe Framework menerapkan arsitektur full-stack web berlapis (*multi-tier architecture*) berbasis Python dan JavaScript dengan MariaDB atau PostgreSQL sebagai mesin basis datanya. Arsitektur ini dirancang untuk memisahkan secara tegas antara lapisan antarmuka pengguna (*presentation layer*), lapisan logika bisnis (*application layer*), dan lapisan penyimpanan data (*persistence layer*), namun tetap terikat secara terintegrasi melalui prinsip *metadata-driven*.

Secara teknis, lapisan-lapisan utama dalam arsitektur Frappe Framework dapat diuraikan sebagai berikut:

1. Lapisan Antarmuka Pengguna (*Client/Presentation Layer*): Antarmuka utama Frappe Framework dibagi menjadi dua komponen:
   - Frappe Desk (Admin Interface): Berupa Single Page Application (SPA) berbasis JavaScript dan HTML/CSS yang dinamis. Tampilan ini berfungsi sebagai pusat kendali (command center) bagi pengguna internal/karyawan untuk mengelola formulir, grafik, laporan, dan alur kerja harian.
   - Portal Web (*Website/Portal Pages*): Tampilan halaman web publik berbasis server-side rendering menggunakan template engine Jinja. Komponen ini diperuntukkan bagi pengguna luar (website users) seperti pelanggan atau pemasok untuk mengakses layanan portal tanpa memerlukan hak akses ke Frappe Desk

2. Lapisan Logika Bisnis & Aplikasi (*Server/Application Layer*): Sisi server dijalankan menggunakan bahasa pemrograman Python. Setiap pemrosesan transaksi dan aturan bisnis dikendalikan oleh Server Controller yang mewarisi kelas dasar Document dari ORM bawaan Frappe. Lapisan ini juga mengelola komunikasi *real-time* berbasis Socket.io, penanganan tugas latar belakang (*background jobs*) melalui Redis dan RQ worker, serta penyediaan layanan REST API terenkripsi (*token-based authentication*) untuk integrasi dengan aplikasi eksternal.

3. Lapisan Penyimpanan Data (*Database/Persistence Layer*): Seluruh data transaksi dan konfigurasi metadata disimpan di dalam *relational database* MariaDB atau PostgreSQL. Ketika sebuah DocType baru dibuat di sistem, Frappe ORM secara otomatis merelasikannya ke dalam tabel fisik database dengan menambahkan awalan tab pada

nama tabel tersebut (contoh: DocType Sales Invoice disimpan dalam tabel tabSales Invoice). Selain itu, Redis digunakan sebagai memori tembolok (*caching*) untuk mempercepat kueri data dan manajemen sesi pengguna.

Abstraksi data dalam Frappe Framework menerapkan pendekatan arsitektur berbasis metadata (*metadata-driven architecture*) yang menyembunyikan kompleksitas teknis pengelolaan basis data tingkat rendah seperti eksekusi kueri SQL, pembuatan tabel fisik, dan prosedur migrasi skema. Melalui konsep tunggal bernama DocType, kerangka kerja ini menyatukan definisi skema basis data, aturan validasi bisnis, tata letak antarmuka pengguna di Frappe Desk, hingga penyediaan endpoint REST API ke dalam satu konfigurasi terpadu. Pendekatan ini memungkinkan pengembang untuk berinteraksi dengan data melalui lapisan ORM (*Object-Relational Mapping*) berbasis Python dan objek dokumen bisnis secara intuitif, sehingga perubahan struktur data pada tingkat aplikasi dapat langsung dipetakan ke dalam tabel fisik MariaDB atau PostgreSQL secara otomatis tanpa memerlukan penyelarasan manual antar-lapisan.

Pada tingkat penyajian dan integrasi, abstraksi data ini dikembangkan lebih lanjut melalui berbagai variasi model DocType yang disesuaikan dengan kebutuhan arsitektur sistem, seperti Standard DocType untuk tabel transaksi relasional, Single DocType untuk penyimpanan konfigurasi berbasis key-value, Child DocType untuk struktur data berulang, serta Virtual DocType yang mengabstraksikan sumber data eksternal tanpa menulis ke basis data lokal. Selain itu, hadirnya Virtual DocField memungkinkan kalkulasi data dilakukan secara dinamis di memori lapisan aplikasi saat runtime tanpa mengotori skema basis data fisik. Dengan menyelimuti seluruh proses dari penyimpanan fisik, pengolahan logika backend, hingga tampilan antarmuka dan API dalam satu abstraksi yang utuh, Frappe Framework berhasil memangkas beban kerja pengembangan secara signifikan sekaligus menjaga konsistensi serta integritas data di seluruh ekosistem aplikasi.

Abstraksi data di dalam arsitektur Frappe dikelompokkan ke dalam empat model DocType utama untuk mengoptimalkan kinerja basis data:

- Standard/Document DocType: Memetakan entitas transaksi atau master data ke dalam satu tabel fisik basis data, di mana setiap dokumen disimpan sebagai satu baris (*row*).

- Single DocType: Digunakan khusus untuk panel konfigurasi dan dashboard tanpa membuat tabel baru tersendiri; seluruh nilainya disimpan dalam bentuk pasangan key-value global.
- Child/Table DocType: Tabel anak yang tertaut pada dokumen induk (*parent record*) untuk menyimpan array data berulang, seperti rincian line items pada faktur penjualan.
- Virtual DocType: Model dokumen yang tidak menuliskan data ke basis data lokal, melainkan mengambil dan mengirim data langsung dari sumber eksternal melalui pengontrol Python custom.

Dalam tata kelola sistem, seluruh lingkungan multi-tenant dan repositori aplikasi dikelola secara terpusat menggunakan Bench CLI. Frappe menerapkan pemisahan yang ketat antara berkas kerangka kerja utama (*core framework*) dengan aplikasi kustom (*custom apps*). Seluruh kustomisasi dikemas ke dalam custom app terpisah yang dihubungkan melalui skrip hooks.py, sehingga kode inti sistem tetap bersih, aman saat dilakukan pembaruan (*upgrade*), serta terisolasi secara modular.

# BAB II
# INSTALASI & KONFIGURASI LINGKUNGAN FRAPPE

## 2.1 Persiapan Lingkungan Virtualisasi Windows dan WSL 2

Persiapan lingkungan instalasi diawali dengan memastikan bahwa sistem operasi Windows telah memenuhi prasyarat virtualisasi perangkat keras, di mana fitur Intel VT-x atau AMD-V harus berada dalam status aktif melalui pengaturan BIOS/UEFI dan dapat diverifikasi pada menu *Task Manager* bagian CPU. Selanjutnya, pengelolaan virtualisasi Linux dijalankan dengan memperbarui komponen kernel *Windows Subsystem* for Linux (WSL) ke versi 2 menggunakan perintah `wsl --update` pada PowerShell berhak akses Administrator. Kepastian bahwa distribusi Linux Ubuntu yang terpasang berjalan di atas arsitektur WSL 2 divalidasi melalui perintah `wsl -l -v`, di mana versi 2 menawarkan integrasi performa kernel Linux murni yang jauh lebih efisien dalam mengelola kontainer.

Untuk menghindari penurunan kinerja perangkat akibat konsumsi memori berlebih oleh proses mesin virtual WSL (`Vmmem`), batasan alokasi perangkat keras dikonfigurasi melalui pembentukan berkas `.wslconfig` di direktori profil pengguna `%UserProfile%` (`C:\Users\<Username>\.wslconfig`). Konfigurasi pembatasan ini dituliskan sebagai berikut:

```powershell
[wsl2]
memory=4GB
processors=4
swap=2GB
autoMemoryReclaim=gradual
```

Berkas konfigurasi di atas membatasi kapasitas RAM fisik maksimum sebesar 4GB, mengalokasikan 4 inti prosesor, memori virtual swap sebesar 2GB, serta mengaktifkan pembebasan memori secara bertahap melalui fitur `autoMemoryReclaim`. Setelah berkas pembatas disimpan, mesin WSL dihentikan secara menyeluruh menggunakan perintah `wsl --shutdown` pada terminal agar seluruh alokasi sumber daya baru dapat diterapkan secara presisi sebelum layanan kontainer mulai dijalankan.

### 2.2 Instalasi dan Konfigurasi Docker Desktop

Pengelolaan lingkungan kontainer ERPNext bertumpu pada aplikasi Docker Desktop untuk Windows yang dipasang dengan mengaktifkan opsi *Use WSL 2 instead of Hyper-V* pada proses instalasi. Pilihan subsistem ini memungkinkan *Docker Engine* berkomunikasi langsung dengan kernel Linux Ubuntu di WSL 2 secara efisien tanpa beban virtualisasi Hyper-V yang berat. Setelah aplikasi Docker Desktop berhasil berjalan dan menunjukkan status *Engine running*, integrasi layanan dikonfigurasi melalui menu *Settings* > *Resources* > *WSL integration*, di mana pengembang mengaktifkan integrasi langsung ke distribusi utama Ubuntu agar perintah CLI Docker dapat dikenali secara native di terminal Linux.

Agar pengguna biasa di Linux Ubuntu memiliki hak akses penuh dalam mengendalikan *daemon* Docker tanpa hambatan *permission denied*, akun pengguna dimasukkan ke dalam grup sistem `docker` dan izin *socket* diperbarui melalui baris perintah berikut:

```powershell
# Menambahkan user ke grup docker dan mengonfigurasi izin socket
sudo usermod -aG docker $USER
sudo chmod 666 /var/run/docker.sock

# Memverifikasi hak akses docker tanpa sudo
docker ps
```

Pengoperasian perintah di atas memperbarui izin baca-tulis pada socket komunikasi Docker `/var/run/docker.sock`. Keberhasilan seluruh konfigurasi hak akses divalidasi ketika perintah `docker ps` dapat dieksekusi tanpa tambahan `sudo`, yang menandakan bahwa lingkungan runtime kontainer telah siap mengelola citra aplikasi.

### 2.3 Pengunduhan dan Penyiapan Repositori frappe_docker

Prinsip utama dalam menjaga kinerja baca-tulis sistem berkas (*disk I/O performance*) pada WSL 2 adalah menempatkan seluruh direktori kerja proyek di dalam *Home Directory Linux* murni (`/home/<username>/` atau `~`) dan bukan di dalam direktori Windows (`/mnt/c/`). Penempatan berkas di dalam sistem berkas Linux murni mencegah latensi lambat pada jembatan antarmuka WSL saat sistem ERPNext melakukan operasi basis data dan pembacaan ribuan berkas aplikasi web.

Proses penyiapan repositori dilakukan dari terminal Ubuntu WSL 2 melalui langkah-langkah perintah berikut:

```powershell
# Pindah ke Home Directory Linux murni
cd ~

# Mengunduh repositori resmi frappe_docker dari GitHub
git clone https://github.com/frappe/frappe_docker.git

# Masuk ke direktori kerja frappe_docker
cd ~/frappe_docker
```

Repositori `frappe_docker` menyediakan seluruh resep orkestrasi Docker Compose, konfigurasi jaringan virtual, dan skrip instalasi yang dirancang resmi oleh tim pengembang Frappe Technologies. Setelah proses pengunduhan selesai dan pengguna masuk ke direktori `~/frappe_docker`, tahap deployment kontainer siap dilaksanakan.

### 2.4 Deployment Container ERPNext Menggunakan Docker Compose

Proses *deployment* lingkungan ERPNext lokal dijalankan dengan mengarsitekturkan beraneka layanan kontainer melalui berkas `pwd.yml` (*Play-with-Docker*) menggunakan perintah orkestrasi Docker Compose. Langkah eksekusi dan pemantauan inisialisasi sistem dilakukan melalui baris perintah berikut:

```powershell
# Menjalankan seluruh kontainer ERPNext dalam mode detached (-d)
docker compose -f pwd.yml up -d

# Memantau proses inisialisasi dan migrasi database secara real-time
docker compose -f pwd.yml logs -f create-site
```

Eksekusi perintah `up -d` mengunduh citra kontainer resmi dari Docker Hub yang mencakup MariaDB 11.8 sebagai basis data relasional, Redis 6.2-alpine sebagai pengelola cache dan antrean tugas, serta citra utama `frappe/erpnext`. Pemantauan log lewat perintah `logs -f create-site` memperlihatkan proses migrasi otomatis ratusan DocType hingga pemuatan

dasbor aplikasi, di mana pemunculan pesan *Current Site set to frontend* menjadi indikator bahwa struktur tabel basis data telah terbentuk sempurna.

### 2.5 Pengaksesan Sistem dan Pengisian Setup Wizard

Sistem ERPNext yang telah aktif di dalam lingkungan kontainer dapat diakses oleh pengguna melalui peramban web pada sistem operasi Windows. Informasi pengaksesan dan kredensial log masuk awal sistem dikonfigurasi sebagai berikut:

```powershell
URL Pengaksesan Web UI : http://localhost:8080/app
Username Administrator : Administrator
Password Default     : admin
```

Pada halaman log masuk awal, autentikasi dilakukan menggunakan kredensial administrator bawaan di atas. Apabila terdapat kendala peramban yang mengalami *redirect* otomatis ke alamat internal kontainer `http://frontend:8080` sehingga halaman tidak dapat dijangkau, pengguna cukup memperbarui bilah URL dengan mengganti kata `frontend` menjadi `localhost` sehingga menjadi `http://localhost:8080/app`.

Langkah selanjutnya adalah menyelesaikan formulir panduan awal (*Setup Wizard*) yang berfungsi membentuk profil dasar organisasi dan akun pembukuan. Pengguna menentukan pilihan bahasa antarmuka dan memilih negara Indonesia, yang secara otomatis mengonfigurasi zona waktu lokal ke Asia/Jakarta serta menetapkan mata uang standar ke IDR (Rupiah). Setelah memperbarui kata sandi administrator dan memasukkan identitas perusahaan serta menentukan periode tahun fiskal, sistem akan menyelesaikan konfigurasi akhir dan mengarahkan pengguna ke antarmuka utama Frappe Desk.

### 2.6 Operasional Harian dan Prosedur Reset Sistem

Pengelolaan operasional harian ERPNext memberikan fleksibilitas bagi pengguna untuk menghentikan dan menjalankan kembali layanan kontainer sesuai kebutuhan penggunaan perangkat komputer. Perintah-perintah operasional harian dan prosedur pembersihan total sistem dikelompokkan sebagai berikut:

```powershell
# === OPERASIONAL HARIAN ===
# Menghentikan sementara seluruh kontainer tanpa menghapus
data
docker compose -f pwd.yml stop

# Menyalahkan kembali kontainer yang dihentikan
docker compose -f pwd.yml start

# Memeriksa status kesehatan kontainer
docker compose -f pwd.yml ps

# === PROSEDUR RESET TOTAL (START FRESH) ===
# Menghapus seluruh kontainer beserta volume basis data
secara permanen
docker compose -f pwd.yml down -v
```

Saat operasional selesai, perintah `stop` digunakan untuk menghentikan kontainer secara tertib sehingga seluruh data transaksi tersimpan aman di dalam volume. Untuk mengaktifkan kembali layanan di hari berikutnya, pengguna cukup mengeksekusi perintah `start`. Apabila terjadi kesalahan konfigurasi fatal atau pengguna ingin membersihkan seluruh data simulasi untuk memulai pengujian dari awal (*fresh install*), prosedur reset total dilakukan dengan mengeksekusi perintah `down -v`. Opsi flag `-v` secara permanen memusnahkan volume basis data lama sehingga sistem dapat dibangun ulang menjadi bersih sempurna.

# BAB III
# KONFIGURASI MASTER DATA PERUSAHAAN MANUFAKTUR

## 3.1 Bagan Akun (Chart of Accounts/COA) Standar Manufaktur 4 Level

Bagan Akun atau *Chart of Accounts* (COA) merupakan tulang punggung sistem akuntansi di ERPNext yang mengelompokkan seluruh transaksi keuangan ke dalam pohon hierarki. Secara bawaan, ERPNext menyediakan struktur hierarki lima *Root Types* utama, yaitu Aset (*Asset*), Kewajiban (*Liability*), Ekuitas (*Equity*), Pendapatan (*Income*), dan Beban (*Expense*). Untuk kebutuhan industri manufaktur, COA dirancang hingga empat tingkatan (*4-level hierarchy*) guna memisahkan akun operasional pabrikasi secara mendetail. Level 1 merepresentasikan akun induk utama (*Root*), Level 2 merepresentasikan kelompok akun (*Group Accounts* seperti Aset Lancar), Level 3 merepresentasikan sub-kelompok (seperti Persediaan), dan Level 4 merepresentasikan akun detail (*Leaf Accounts*) tempat jurnal transaksi dibukukan.

Dalam perusahaan manufaktur, keberadaan akun khusus persediaan dan biaya produksi sangat vital untuk menghitung Harga Pokok Production (HPP) dan Harga Pokok Penjualan secara otomatis. Akun persediaan dibagi menjadi tiga kategori utama, yaitu `1131 - Persediaan Bahan Baku` (*Raw Materials*), `1133 - Persediaan Barang Dalam Proses` (*Work in Progress / WIP*), dan `1134 - Persediaan Barang Jadi` (*Finished Goods*). Sementara itu, pada kelompok biaya, sistem membedakan antara `5120 - Biaya Tenaga Kerja Langsung` (BTKL) dan kelompok `5130 - Biaya Overhead Pabrik` (BOP) seperti tagihan listrik pabrik (`5131`) dan penyusutan mesin (`5132`). Setiap akun detail dikonfigurasi dengan *Account Type* spesifik (seperti *Stock*, *Payable*, *Receivable*, *Bank*, atau *Direct Expense*) yang berfungsi sebagai label identifikasi fungsi kontrol transaksi otomatis di ERPNext.

Pengaturan struktur COA manufaktur 4 level untuk impor masal dapat dipetakan menggunakan struktur JSON/Python sebagai berikut:

```powershell
# Pemetaan Struktur Chart of Accounts Manufaktur 4 Level di ERPNext
coa_structure = {
    "1000 - Aset": {
        "1100 - Aset Lancar": {
            "1130 - Persediaan": {
                "1131 - Persediaan Bahan Baku": {"account_type": "Stock"},
                "1133 - Persediaan Barang Dalam Proses (WIP)": {"account_type": "Stock"},
                "1134 - Persediaan Barang Jadi": {"account_type": "Stock"}
            }
        }
    },
    "5000 - Beban": {
        "5100 - Biaya Produksi": {
            "5110 - Harga Pokok Penjualan (HPP)": {"account_type": "Cost of Goods Sold"},
            "5120 - Biaya Tenaga Kerja Langsung (BTKL)": {"account_type": "Direct Expense"},
            "5130 - Biaya Overhead Pabrik (BOP)": {
                "5131 - Biaya Listrik & Utilitas Pabrik": {"account_type": "Direct Expense"},
                "5132 - Biaya Penyusutan Mesin Pabrik": {"account_type": "Direct Expense"}
            }
        }
    }
}
```

### 3.2 Konfigurasi Master Gudang (Warehouse) dan Hirarki Lokasi

Pengelolaan persediaan fisik di ERPNext diatur melalui struktur hierarki *Warehouse* (Gudang) pada Modul *Stock*. Gudang di ERPNext dibedakan menjadi dua jenis fungsi, yaitu Gudang Induk (*Group Warehouse*) yang berfungsi sebagai wadah pengelompokkan lokasi tanpa menyimpan stok langsung, dan Gudang Detail (*Leaf Warehouse*) yang aktif digunakan sebagai lokasi penyimpanan fisik barang serta terikat dengan pembukuan jurnal akuntansi persediaan (*Stock Ledger*). Struktur gudang yang terorganisasi dengan baik memastikan setiap pergerakan barang mentah, komponen setengah jadi, hingga barang siap jual tercatat secara presisi.

Pada entitas manufaktur secara umum, konfigurasi gudang dipisahkan sesuai dengan tahapan alur produksi. Tiga gudang utama yang umum dibentuk adalah Gudang Bahan Baku (*Raw Materials Warehouse*) untuk menampung komponen yang dibeli dari pemasok, Gudang WIP (*Work in Progress Warehouse*) untuk menampung material yang sedang berada dalam proses perakitan di lantai pabrik, dan Gudang Barang Jadi (*Finished Goods Warehouse*) untuk menyimpan produk akhir yang siap dikirimkan kepada pelanggan. Selain gudang fisik, ERPNext juga mendukung gudang virtual seperti *Goods in Transit* yang digunakan untuk melacak pergerakan transfer barang antar lokasi sebelum barang diterima secara resmi.

Pembentukan gudang manufaktur secara terprogram menggunakan REST API Python di ERPNext dikonfigurasi melalui baris kode umum berikut:

```powershell
import requests

# Konfigurasi pembuatan Master Gudang Manufaktur via REST API ERPNext
BASE_URL = "http://localhost:8080/api/resource/Warehouse"
HEADERS = {
    "Authorization": "token API_KEY:API_SECRET",
    "Content-Type": "application/json"
}

warehouses = [
    {"warehouse_name": "Gudang Bahan Baku", "is_group": 0, "parent_warehouse":
"All Warehouses"},
    {"warehouse_name": "Gudang WIP", "is_group": 0, "parent_warehouse": "All
Warehouses"},
    {"warehouse_name": "Gudang Barang Jadi", "is_group": 0, "parent_warehouse":
"All Warehouses"}
]

for wh in warehouses:
    response = requests.post(BASE_URL, headers=HEADERS, json=wh)
```

### 3.3 Pengelolaan Hak Akses Pengguna dan Peran (Role & Permissions)

Keamanan sistem dan pemisahan tugas (*Segregation of Duties/SoD*) di ERPNext dikelola melalui kombinasi akun Pengguna (*User*), Peran (*Role*), dan Pengelola Izin Peran (*Role Permission Manager*). Setiap pengguna sistem diberikan satu atau beberapa Role spesifik (seperti *Purchase User*, *Stock User*, *Manufacturing User*, atau *Accounts User*) yang menentukan tingkat kewenangan pengguna dalam membaca (*Read*), membuat (*Create*), mengubah (*Write*), menyetujui (*Submit*), membatalkan (*Cancel*), atau menghapus (*Delete*) dokumen transaksi pada suatu *DocType*. Administrator sistem memegang peran tertinggi (*System Manager*) dengan akses penuh untuk mengatur konfigurasi global dan hak akses seluruh pengguna.

Selain pembatasan berbasis *Role*, ERPNext menyediakan fitur *User Permissions* yang memungkinkan pembatasan akses data secara lebih spesifik hingga tingkat nilai dokumen (*Document-Level Restriction*). Sebagai contoh, seorang operator gudang bahan baku dapat dibatasi aksesnya sehingga hanya dapat melihat transaksi yang terikat dengan Gudang Bahan Baku, atau seorang staf penjualan hanya dapat mengakses data pelanggan di wilayah operasional tertentu. Pengaturan hak akses berbasis tingkat kolom (*Perm Level*) juga dapat diterapkan untuk menyembunyikan kolom sensitif, seperti nilai kerahasiaan harga pokok produksi dari tampilan operator lantai pabrik.

Pengaturan batasan *User Permission* untuk mengunci hak akses pengguna ke gudang spesifik dikonfigurasi secara umum melalui skrip Python berikut:

```powershell
import frappe

# Menambahkan User Permission khusus untuk membatasi gudang pengguna
def set_warehouse_user_permission(user_email, warehouse_name):
    doc = frappe.get_doc({
        "doctype": "User Permission",
        "user": user_email,
        "allow": "Warehouse",
        "for_value": warehouse_name,
        "apply_to_all_doctypes": 1
    })
    doc.insert(ignore_permissions=True)
    frappe.db.commit()

# Contoh penerapan umum: Pengguna hanya dapat mengakses Gudang Bahan Baku
set_warehouse_user_permission("operator.gudang@perusahaan.com", "Gudang Bahan Baku")
```

### 3.4 Impor Masal Master Data Menggunakan Data Import Tool

Tahap krusial dalam migrasi dari sistem lama ke ERPNext adalah pengunggahan data master secara masal menggunakan *Data Import Tool*. Alat ini mendukung pengunggahan data berformat CSV atau Excel untuk berbagai *DocType* utama seperti *Chart of Accounts*, *Item*, *Supplier*, dan *Customer*. *Data Import Tool* menyediakan dua mode pengoperasian utama, yaitu mode *Insert New Records* untuk memasukkan baris data master baru ke dalam sistem, dan mode *Update Existing Records* untuk memperbarui informasi master data yang sudah terdaftar sebelumnya menggunakan kunci identifikasi unik (*Name / ID*).

Prosedur impor masal diawali dengan mengunduh *Template CSV* resmi dari antarmuka *Data Import* yang memuat nama kolom teknis (*fieldnames*) bawaan DocType. Setelah berkas data diisi, pengguna mengunggah berkas tersebut dan melakukan pemetaan kolom (*column mapping*) untuk mencocokkan setiap judul kolom pada berkas dengan kolom tujuan di ERPNext. Sebelum data dimasukkan ke dalam basis data fisik, sistem secara otomatis mengeksekusi validasi aturan bisnis, seperti verifikasi tipe data, kepastian nilai wajib (*mandatory fields*), serta keberadaan relasi *Link Field* (misalnya memastikan nama kelompok barang atau satuan UOM sudah terdaftar di sistem) guna mencegah terjadinya korupsi skema data.

Format struktur berkas CSV untuk impor masal Master Data Barang (*Item Master*) dikonfigurasi secara umum sebagai berikut:

```powershell
Item Code,Item Name,Item Group,Default Unit of
Measure,Maintain Stock,Is Purchase Item,Is Sales Item
RAW-MAT-01,Komponen Bahan Baku A,Raw Material,Nos,1,1,0
FG-PROD-01,Produk Barang Jadi X,Products,Nos,1,0,1
```

# BAB IV
# OPERASIONAL ENTRY DATA TRANSAKSI BISNIS

## 4.1 Modul Pembelian dan Pengadaan Barang

Modul Pembelian (*Buying Module*) pada ERPNext mengelola seluruh alur pengadaan barang dan jasa, mulai dari pengajuan kebutuhan internal hingga pengakuan kewajiban pembayaran kepada pemasok (*Procurement-to-Pay Cycle*). Siklus ini diawali dengan penerbitan *Material Request* oleh departemen internal atau sistem reorder otomatis ketika stok barang mencapai batas minimum. Dokumen tersebut kemudian diteruskan ke departemen pengadaan untuk diterbitkan *Purchase Order* (PO) resmi kepada pemasok terdaftar. Setelah pemasok mengirimkan barang, bagian penerimaan mencatat fisik barang yang masuk melalui *Purchase Receipt*, yang secara otomatis memperbarui saldo persediaan di gudang. Tahap akhir siklus pengadaan ditutup dengan pembukuan *Purchase Invoice*, yang mengakui timbulnya Hutang Usaha (*Accounts Payable*) pada sistem akuntansi secara otomatis.

Integrasi antar dokumen pada modul pembelian memastikan kontrol internal yang ketat melalui mekanisme *Three-Way Matching* (pencocokan antara PO, Penerimaan Barang, dan Faktur Tagihan). Ketika *Purchase Receipt* disetujui (Submit), ERPNext secara otomatis membukukan transaksi persediaan sementara (*Stock Received But Not Billed*) guna mencatat penambahan barang fisik sebelum faktur resmi diterima. Saat *Purchase Invoice* diterbitkan dan disetujui, akun persediaan sementara tersebut ditutup dan dipindahkan menjadi akun Hutang Usaha resmi. Seluruh alur ini meminimalkan risiko selisih harga atau jumlah barang yang ditagih oleh pemasok.

Pembuatan dokumen *Purchase Order* secara terprogram menggunakan REST API Python di ERPNext dapat dikonfigurasi melalui baris kode berikut:

```powershell
import requests

# Konfigurasi pembuatan Purchase Order via REST API ERPNext
BASE_URL = "http://localhost:8080/api/resource/Purchase Order"
HEADERS = {
    "Authorization": "token API_KEY:API_SECRET",
    "Content-Type": "application/json"
}

po_payload = {
    "supplier": "PT Pemasok Utama",
    "schedule_date": "2026-10-20",
    "items": [
        {
            "item_code": "RAW-MAT-01",
            "qty": 100,
            "rate": 50000,
            "warehouse": "Gudang Bahan Baku - PERUS"
        }
    ]
}

response = requests.post(BASE_URL, headers=HEADERS, json=po_payload)
```

### 4.2 Modul Penjualan dan Pengelolaan Pelanggan

Modul Penjualan (*Selling Module*) di ERPNext memfasilitasi seluruh rantai pendapatan perusahaan dari tahap prospek awal hingga penerimaan kas (*Order-to-Cash Cycle*). Alur diawali dengan pengelolaan data calon pelanggan (*Lead*) dan peluang bisnis (*Opportunity*), yang kemudian ditindaklanjuti dengan penerbitan Surat Penawaran Harga (*Quotation*). Apabila penawaran disetujui oleh pelanggan, sistem mengkonversinya menjadi *Sales Order* sebagai komitmen pemesanan resmi. Selanjutnya, bagian logistik mengkonfirmasi pengeluaran fisik barang dari gudang melalui *Delivery Note*, yang secara otomatis mengurangi stok persediaan dan mengakui nilai Harga Pokok Penjualan (HPP). Penagihan dilakukan dengan membuat *Sales Invoice* untuk membukukan Piutang Usaha (*Accounts Receivable*) dan Pendapatan Penjualan.

Sistem pergerakan data otomatis pada modul penjualan memastikan bahwa setiap transaksi terpaut secara konsisten dari dokumen induk ke dokumen turunan. Saat *Delivery Note* dan *Sales Invoice* diterbitkan dari *Sales Order*, ERPNext secara otomatis menghitung HPP berdasarkan metode penilaian stok yang dikonfigurasi (seperti FIFO atau *Moving Average*). Selain itu, sistem mendukung skenario penagihan parsial maupun pengiriman bertahap dengan memperbarui status pemenuhan (*Status Indicator*) pada *Sales Order* secara real-time, sehingga manajemen penjualan dapat memantau pesanan yang belum terkirim (*backorder*) secara presisi.

Otomatisasi pembuatan *Sales Order* dari *Quotation* menggunakan REST API Python dikonfigurasi melalui skrip berikut:

```powershell
import requests

# Konfigurasi pembuatan Sales Order dari Quotation via REST API
BASE_URL = "http://localhost:8080/api/resource/Sales Order"
HEADERS = {
    "Authorization": "token API_KEY:API_SECRET",
    "Content-Type": "application/json"
}

so_payload = {
    "customer": "PT Pelanggan Utama",
    "delivery_date": "2026-10-25",
    "items": [
        {
            "item_code": "FG-PROD-01",
            "qty": 50,
            "rate": 120000,
            "warehouse": "Gudang Barang Jadi - PERUS"
        }
    ]
}

response = requests.post(BASE_URL, headers=HEADERS, json=so_payload)
```

### 4.3 Modul Persediaan Stok dan Operasional Manufaktur

Modul Stok dan Manufaktur mengatur alur transformasi bahan mentah menjadi produk jadi melalui pengorganisasian data master barang (*Item Master*), Resep Produksi (*Bill of Materials/BOM*), Perintah Kerja (*Work Order*), dan Pencatatan Stok (*Stock Entry*). Master barang mengidentifikasi peran item sebagai bahan baku (*Is Purchase Item*) atau barang jadi (*Is Sales Item*). BOM bertindak sebagai fondasi utama yang mendefinisikan daftar komponen bahan baku beserta takaran kuantitas dan alokasi biaya operasi yang dibutuhkan untuk menghasilkan satu satuan unit barang jadi. BOM ini menjadi acuan utama sistem dalam menghitung kebutuhan material (*Material Requirements Planning/MRP*).

Proses eksekusi produksi dimulai dengan penerbitan dokumen *Work Order* untuk menetapkan jumlah target produksi dan jadwal pengerjaan. Pelaksanaan perakitan di lantai pabrik melibatkan dua tahap *Stock Entry* utama, yaitu *Material Issue for Manufacture* untuk memindahkan bahan baku dari Gudang Bahan Baku ke Gudang WIP (*Work in Progress*), serta *Manufacture Entry* saat proses produksi selesai untuk memindahkan barang jadi ke Gudang Barang Jadi. Pada tahap penyelesaian produksi, ERPNext secara otomatis mengakumulasikan

biaya bahan baku dan tambahan biaya operasional (seperti BTKL dan BOP) ke dalam nilai harga pokok per unit barang jadi yang baru terbentuk.

Pengisian dokumen *Work Order* secara terprogram untuk memulai proses produksi dikonfigurasi melalui baris kode berikut:

```powershell
import requests

# Konfigurasi pembuatan Work Order Manufaktur via REST API
BASE_URL = "http://localhost:8080/api/resource/Work Order"
HEADERS = {
    "Authorization": "token API_KEY:API_SECRET",
    "Content-Type": "application/json"
}

wo_payload = {
    "production_item": "FG-PROD-01",
    "bom_no": "BOM-FG-PROD-01-001",
    "qty": 100,
    "wip_warehouse": "Gudang WIP - PERUS",
    "fg_warehouse": "Gudang Barang Jadi - PERUS"
}

response = requests.post(BASE_URL, headers=HEADERS, json=wo_payload)
```

### 4.4 Modul Akuntansi dan Keuangan Operasional

Modul Akuntansi dan Keuangan berfungsi sebagai pusat pencatatan akhir seluruh dampak finansial dari transaksi operasional di ERPNext. Meskipun sebagian besar jurnal pembukuan (seperti faktur pembelian, penjualan, dan penyesuaian nilai persediaan) terbentuk secara otomatis oleh sistem melalui konsep *Continuous Inventory System*, pencatatan transaksi kas non-operasional, penyesuaian akhir bulan, dan alokasi biaya dilakukan melalui *Journal Entry*. *Journal Entry* mendukung berbagai tipe transaksi seperti *Bank Entry*, *Cash Entry*, *Credit Note*, *Debit Note*, serta *Journal Entry* umum untuk mencatat penyusutan aset dan alokasi beban overhead.

Penyelesaian kewajiban pembayaran dan penerimaan Kas/Bank dari pelanggan atau kepada pemasok dikelola secara khusus melalui dokumen *Payment Entry*. *Payment Entry* memfasilitasi rekonsiliasi pembayaran dengan menghubungkan secara langsung transaksi penerimaan kas ke dokumen faktur piutang (*Sales Invoice*) atau pembayaran kas ke faktur hutang

(*Purchase Invoice*). Sistem secara otomatis memperbarui saldo Piutang atau Hutang Usaha serta menghitung selisih kurs (*Exchange Gain/Loss*) jika transaksi dilakukan dalam mata uang asing, sehingga laporan keuangan *Trial Balance* dan *General Ledger* selalu tersaji secara mutakhir dan seimbang (*balanced*).

Pembukuan transaksi pembayaran piutang pelanggan via *Payment Entry* dikonfigurasi secara umum menggunakan skrip Python berikut:

```powershell
import requests

# Konfigurasi pembuatan Payment Entry via REST API
BASE_URL = "http://localhost:8080/api/resource/Payment Entry"
HEADERS = {
    "Authorization": "token API_KEY:API_SECRET",
    "Content-Type": "application/json"
}

payment_payload = {
    "payment_type": "Receive",
    "party_type": "Customer",
    "party": "PT Pelanggan Utama",
    "paid_amount": 6000000,
    "received_amount": 6000000,
    "paid_to": "1112 - Bank Utama - PERUS",
    "references": [
        {
            "reference_doctype": "Sales Invoice",
            "reference_name": "ACC-SINV-2026-00001",
            "allocated_amount": 6000000
        }
    ]
}

response = requests.post(BASE_URL, headers=HEADERS, json=payment_payload)
```

### 4.5 Data Import Tool dan Pengunggahan Data Masal

*Data Import Tool* merupakan fasilitas bawaan ERPNext yang digunakan untuk melakukan migrasi dan pengunggahan data transaksi maupun data master secara masal dalam jumlah besar. Fasilitas ini sangat berguna saat permulaan implementasi sistem (*go-live*) guna menghindari penginputan manual satu per satu yang memakan waktu dan berisiko tinggi terhadap kesalahan pengetikan (*human error*). *Data Import Tool* mendukung format file spreadsheet standar seperti CSV dan XLSX untuk hampir seluruh *DocType* utama di dalam sistem.

Prosedur impor masal diawali dengan pemilihan *DocType* tujuan dan pengunduhan berkas *Template* yang telah memuat struktur kolom bawaan. Pengguna wajib memastikan bahwa nilai pada kolom kunci (*mandatory fields*) serta kolom relasi (*Link Fields*) telah terisi dengan tepat dan sesuai dengan data yang sudah terdaftar di ERPNext. Selama proses pengunggahan berlangsung, sistem melakukan proses validasi skema (*Data Validation*) secara ketat; jika ditemukan baris data yang tidak sesuai atau tidak valid, sistem akan membatalkan seluruh proses transaksi (*rollback*) dan memberikan laporan kesalahan mendetail untuk diperbaiki sebelum diunggah kembali.

Struktur format file CSV untuk impor masal data Master Pemasok (*Supplier Master*) dikonfigurasi secara umum sebagai berikut:

```powershell
Supplier Name,Supplier Group,Supplier Type,Country
PT Pemasok Utama,Raw Material,Company,Indonesia
PT Pemasok Sekunder,Services,Company,Indonesia
```

# BAB V
# PEMBELIAN & PENERIMAAN BAHAN BAKU

## 5.1 Penjelasan Cara Melakukan Fitur di ERPNext

Fitur pengadaan di ERPNext mengelola rantai Procurement-to-Pay secara terintegrasi antara Modul Pembelian (Buying), Modul Stok (Stock), dan Modul Akuntansi (Accounts).

- **Purchase Order (PO)**: Dibuat dari menu **Buying > Purchase Order > New**. Berfungsi sebagai dokumen komitmen pemesanan barang kepada pemasok.
- **Purchase Receipt (PR)**: Dibuat dengan mengklik **Create > Receipt** pada Purchase Order yang telah disetujui (*Submit*). PR berfungsi mencatat fisik barang masuk ke gudang, menambah kuantitas stok di *Stock Ledger*, dan membentuk jurnal persediaan sementara pada *General Ledger*.
- **Purchase Invoice (PI)**: Dibuat dari Purchase Receipt melalui tombol **Create > Invoice**. PI berfungsi membukukan kewajiban Hutang Usaha (*Accounts Payable*) secara resmi dan menutup akun persediaan sementara (*Stock Received But Not Billed*).

## 5.2 Studi Kasus & Latihan Praktikum

PT Borgo Dynamics membeli bahan baku utama berupa 500 unit Komponen Desmo V1 seharga Rp 100.000/unit (Total: Rp 50.000.000) secara kredit dengan termin 30 hari dari pemasok PT Noale Presisi. Barang diterima secara utuh di *Gudang Bahan Baku - BGD* dan dilanjutkan dengan penerbitan faktur tagihan resmi.

1. Langkah Kerja di ERPNext
    - Masuk ke **Buying > Supplier > New**, buat pemasok baru: `PT Noale Presisi`.
    - Masuk ke **Stock > Item > New**, buat item baru: `Komponen Desmo V1` (*Maintain Stock = 1, Default Warehouse = Gudang Bahan Baku - BGD*).
    - Buat **Purchase Order** untuk `PT Noale Presisi` sebanyak 500 unit `@ Rp 100.000`. Klik **Submit**.
    - Pada PO tersebut, klik **Create > Purchase Receipt**, pastikan *Accepted Warehouse* mengarah ke `Gudang Bahan Baku - BGD`. Klik **Submit**.
    - Pada PR tersebut, klik **Create > Purchase Invoice**. Klik **Submit**.

2. Lembar Verifikasi Jurnal Akuntansi (General Ledger)
    - **Debit**: `1131 - Persediaan Bahan Baku` = **Rp 50.000.000**
    - **Kredit**: `2110 - Hutang Usaha (PT Noale Presisi)` = **Rp 50.000.000**

# BAB VI
# PENGELUARAN BAHAN BAKU KE LINI PRODUKSI

## 6.1 Penjelasan Cara Melakukan Fitur di ERPNext

Pengeluaran bahan baku dari gudang penyimpanan menuju lantai pabrik diatur melalui integrasi Modul Manufaktur dan Stok.

- **Bill of Materials (BOM)**: Dibuat dari menu **Manufacturing > Bill of Materials > New**. Menentukan daftar resep bahan baku dan kuantitas standar yang dibutuhkan untuk membuat satu unit barang jadi.
- **Work Order (WO)**: Dibuat dari menu **Manufacturing > Work Order > New**. Menentukan jumlah target barang jadi yang akan diproduksi dan menjadwalkan alokasi material.
- **Stock Entry (Material Transfer for Manufacture)**: Dibuat dari dalam dokumen Work Order dengan mengklik tombol **Start**. Berfungsi memindahkan bahan baku dari Gudang Bahan Baku ke Gudang WIP (*Work in Progress*), yang secara otomatis memperbarui *Stock Ledger* dan mengoreksi saldo persediaan proses pada *General Ledger*.

## 6.2 Studi Kasus & Latihan Praktikum

Tim Produksi PT Borgo Dynamics merencanakan pembuatan 50 unit Engine Pack GP26. Berdasarkan BOM, setiap 1 unit barang jadi memerlukan 4 unit Komponen Desmo V1. Tim memindahkan 200 unit Komponen Desmo V1 (nilai total: Rp 20.000.000) dari *Gudang Bahan Baku - BGD* ke *Gudang WIP - BGD*.

1. Langkah Kerja di ERPNext
    - Masuk ke **Stock > Item > New**, buat item barang jadi: `Engine Pack GP26`.
    - Masuk ke **Manufacturing > BOM > New**, pilih item `Engine Pack GP26`, masukkan bahan baku `Komponen Desmo V1` dengan Qty = 4. Klik **Submit**.
    - Masuk ke **Manufacturing > Work Order > New**, pilih item `Engine Pack GP26` dengan Qty = 50. Klik **Submit**.

- Klik tombol **Start** pada Work Order untuk menghasilkan **Stock Entry** (*Type: Material Transfer for Manufacture*). Pastikan *Source Warehouse* = `Gudang Bahan Baku - BGD` dan *Target Warehouse* = `Gudang WIP - BGD`. Klik **Submit**.

2. Lembar Verifikasi Jurnal Akuntansi (General Ledger)

    - **Debit**: `1133 - Persediaan Barang Dalam Proses (WIP)` = **Rp 20.000.000**
    - **Kredit**: `1131 - Persediaan Bahan Baku` = **Rp 20.000.000**

# BAB VII
# PENCATATAN BIAYA TENAGA KERJA LANGSUNG & OVERHEAD PABRIK

## 7.1 Penjelasan Cara Melakukan Fitur di ERPNext

Biaya operasional pabrikasi yang tidak berasal dari transaksi pembelian persediaan langsung dicatat melalui Modul Akuntansi.

- **Journal Entry (Bank Entry)**: Dibuat dari menu **Accounting > Journal Entry > New**. Digunakan untuk membukukan pengeluaran kas/bank atas Biaya Tenaga Kerja Langsung (BTKL) dan Biaya Overhead Pabrik (BOP) seperti utilitas dan listrik pabrik. Jurnal ini secara langsung membebani akun biaya produksi pada Laba Rugi sebelum dialokasikan ke nilai produk jadi.

## 7.2 Studi Kasus & Latihan Praktikum

PT Borgo Dynamics membayar gaji teknisi perakitan pabrik sebesar Rp 8.000.000 dan tagihan listrik operasional mesin sebesar Rp 4.000.000 melalui transfer rekening Bank Mandiri Utama.

1. Langkah Kerja di ERPNext
    - Masuk ke **Accounting > Journal Entry > New**.
    - Set *Entry Type* = `Bank Entry`.
    - Baris 1: Debit `5120 - Biaya Tenaga Kerja Langsung (BTKL)` = **Rp 8.000.000**.
    - Baris 2: Debit `5131 - Biaya Listrik & Utilitas Pabrik` = **Rp 4.000.000.**
    - Baris 3: Kredit `1112 - Bank Mandiri Utama` = **Rp 12.000.000.**
    - Klik **Submit**.

2. Lembar Verifikasi Jurnal Akuntansi (General Ledger)
    - **Debit**: `5120 - BTKL` = **Rp 8.000.000**
    - **Debit**: `5131 - Biaya Listrik & Utilitas Pabrik` = **Rp 4.000.000**

- **Kredit**: `1112 - Bank Mandiri Utama` = **Rp 12.000.000**

# BAB VIII
# PENYELESAIAN PRODUKSI & PENGAKUAN BARANG JADI

## 8.1 Penjelasan Cara Melakukan Fitur di ERPNext

Tahap akhir produksi mencatat pengakuan barang jadi dan kapitalisasi seluruh biaya produksi ke dalam nilai persediaan fisik.

- S**tock Entry (Manufacture)**: Dibuat dengan mengklik tombol **Finish** pada dokumen Work Order. Fitur ini menarik bahan baku dari Gudang WIP, memfasilitasi pengisian tabel *Additional Costs* (BTKL dan BOP), serta menghitung **Valuation Rate** (Harga Pokok Production per unit) barang jadi secara otomatis saat barang ditransfer ke Gudang Barang Jadi.

## 8.2 Studi Kasus & Latihan Praktikum

PT Borgo Dynamics menyelesaikan perakitan 50 unit *Engine Pack GP26*. Total biaya produksi yang diserap adalah Rp 32.000.000 (Bahan Baku Rp 20jt + BTKL Rp 8jt + BOP Rp 4jt), sehingga HPP per unit tercatat sebesar Rp 640.000/unit. Barang ditransfer dari *Gudang WIP - BGD* ke *Gudang Barang Jadi - BGD.*

1. Langkah Kerja di ERPNext
    - Buka dokumen **Work Order** (dari Bab 6), klik tombol **Finish** untuk menerbitkan **Stock Entry** (*Type: Manufacture*).
    - Pada tabel *Additional Costs*, tambahkan dua baris:
        - Baris 1: *Account* = `5120 - BTKL`, *Amount* = `8000000`.
        - Baris 2: *Account* = `5131 - Biaya Listrik Pabrik`, *Amount* = `4000000`.
    - Pastikan *Target Warehouse* = `Gudang Barang Jadi - BGD`.
    - Klik **Submit**.

2. Lembar Verifikasi Jurnal Akuntansi (General Ledger)
    - **Debit**: `1134 - Persediaan Barang Jadi` = **Rp 32.000.000**
    - **Kredit**: `1133 - Persediaan Barang Dalam Proses (WIP)` = **Rp 20.000.000**

- **Kredit**: `5120 - Biaya Tenaga Kerja Langsung (Alokasi)` = **Rp 8.000.000**
- **Kredit**: `5131 - Biaya Listrik & Utilitas Pabrik (Alokasi)` = **Rp 4.000.000**

# BAB IX
# PENJUALAN BARANG JADI & PENGAKUAN HARGA POKOK PENJUALAN

## 9.1 Penjelasan Cara Melakukan Fitur di ERPNext

Siklus Order-to-Cash mengelola penjualan produk hingga pengakuan HPP secara otomatis melalui Perpetual Inventory System.

- **Sales Order (SO)**: Dibuat dari **Selling > Sales Order > New**. Menjadi komitmen pemesanan pelanggan.
- **Delivery Note (DN)**: Dibuat dari SO via **Create > Delivery**. Berfungsi mengeluarkan fisik barang dari gudang dan mencatat pengakuan Harga Pokok Penjualan (HPP) pada *General Ledger*.
- **Sales Invoice (SI)**: Dibuat dari DN via **Create > Invoice**. Mengakui timbulnya Piutang Usaha (*Accounts Receivable*) dan Pendapatan Penjualan.

## 9.2 Studi Kasus & Latihan Praktikum

PT Borgo Dynamics menjual 40 unit Engine Pack GP26 secara kredit (termin 14 hari) kepada pelanggan PT Misano Velocita seharga Rp 1.200.000/unit (Total Penjualan: Rp 48.000.000). Nilai HPP barang yang dikeluarkan adalah Rp 25.600.000 (40 unit x Rp 640.000).

1. Langkah Kerja di ERPNext
    - Masuk ke **Selling > Customer > New**, buat pelanggan: `PT Misano Velocita`.
    - Buat **Sales Order** untuk `PT Misano Velocita` sebanyak 40 unit `Engine Pack GP26` `@ Rp 1.200.000`. Klik **Submit**.
    - Pada SO, klik **Create > Delivery Note**. Pastikan *Source Warehouse* = `Gudang Barang Jadi - BGD`. Klik **Submit**.
    - Pada DN, klik **Create > Sales Invoice**. Klik **Submit**.

2. Lembar Verifikasi Jurnal Akuntansi (General Ledger)
    - **Debit**: `1121 - Piutang Usaha (PT Misano Velocita)` = **Rp 48.000.000**

- **Kredit**: `4110 - Penjualan Barang Jadi` = **Rp 48.000.000**
- **Debit**: `5110 - Harga Pokok Penjualan (HPP)` = **Rp 25.600.000**
- **Kredit**: `1134 - Persediaan Barang Jadi` = **Rp 25.600.000**

# BAB X
# PELUNASAN PIUTANG & JURNAL PENYESUAIAN PENYUSUTAN MESIN

## 10.1 Penjelasan Cara Melakukan Fitur di ERPNext

Bab penutup ini mengelola penyelesaian pembayaran piutang serta penyesuaian akuntansi akhir bulan.

- **Payment Entry**: Dibuat langsung dari Sales Invoice melalui tombol **Create > Payment**. Berfungsi membukukan penerimaan kas/bank dan melakukan rekonsiliasi piutang (*Payment Reconciliation*).
- **Adjusting Journal Entry**: Dibuat dari **Accounting > Journal Entry > New** untuk membukukan pembebanan non-kas seperti penyusutan aset tetap pabrik pada akhir periode.

## 10.2 Studi Kasus & Latihan Praktikum

- Transaksi A: Pelanggan PT Misano Velocita melunasi seluruh tagihan piutangnya sebesar Rp 48.000.000 via transfer Bank Mandiri Utama.
- Transaksi B: PT Borgo Dynamics mencatat jurnal penyesuaian penyusutan mesin produksi bulan berjalan sebesar Rp 1.500.000.

1. Langkah Kerja di ERPNext
    - Buka dokumen **Sales Invoice** (dari Bab 9), klik tombol **Create > Payment**.
    - Pastikan *Account Paid To* = `1112 - Bank Mandiri Utama`. Klik **Submit**.
    - Masuk ke **Accounting > Journal Entry > New** (*Type: Journal Entry*):
        - **Debit**: `5132 - Biaya Penyusutan Mesin Pabrik` = **Rp 1.500.000**.
        - **Kredit**: `1229 - Akumulasi Penyusutan Mesin Pabrik` = **Rp 1.500.000.**
    - Klik **Submit**.

2. Lembar Verifikasi Jurnal Akuntansi (General Ledger)

- Transaksi A (Pelunasan Piutang):
    - **Debit**: `1112 - Bank Mandiri Utama` = **Rp 48.000.000**
    - **Kredit**: `1121 - Piutang Usaha (PT Misano Velocita)` = **Rp 48.000.000**
- Transaksi B (Penyusutan Mesin):
    - **Debit**: `5132 - Biaya Penyusutan Mesin Pabrik` = **Rp 1.500.000**
    - **Kredit**: `1229 - Akumulasi Penyusutan Mesin Pabrik` = **Rp 1.500.000**